Lab 7 — Fuzzing for Input Validation Bugs

PES1UG24AM427 | Dhanush R | Software Engineering Lab

Module	car_insurance_premium_parser.py
Test file	test_car_insurance_premium_parser.py
Testing framework	Hypothesis (property-based fuzzing)
Python version	3.12.7
Date	11 November 2026

1. Bugs Found in the Original Module

#	Function	Input that crashed it	Exception raised	Root cause
1	normalize_registration()	None	AttributeError: 'NoneType' has no attribute 'strip'	No type check before calling .strip()
2	normalize_registration()	'' (empty string) or '.' (punct-only)	Returns '' silently (empty result — logic gap)	No guard after stripping separators leaves empty plat
3	total_claim_amount()	None	AttributeError: 'NoneType' has no attribute 'split'	No type check before calling .split()
4	total_claim_amount()	'abc;xyz' (non-numeric tokens)	ValueError: invalid literal for int()	int() called blindly on every token without validation
5	driver_age()	None	AttributeError: 'NoneType' has no attribute 'split'	No type check before calling .split()
6	driver_age()	'not-a-date' (too few '/' separators)	IndexError: list index out of range	No check that split('/') yields exactly 3 parts
7	driver_age()	'14/08/YYYY' (non-numeric year field)	ValueError: invalid literal for int()	int() called on year field without digit validation

2. Fixes Applied

normalize_registration()

• Added isinstance(plate, str) guard → raises ValueError for non-strings.

• Added blank-check after strip() → raises ValueError for whitespace-only input.

• Added post-removal blank-check → raises ValueError for punctuation-only strings like '.'.

total_claim_amount()

• Added isinstance(text, str) guard → raises ValueError for non-strings.

• Added blank-check after strip() → raises ValueError for empty/whitespace input.

• Each semicolon-separated token is validated with isdigit() before int() conversion;

• non-numeric tokens raise ValueError with the offending token shown.

driver_age()

• Added isinstance(dob, str) guard → raises ValueError for non-strings.

• Checked that split('/') yields exactly 3 parts → raises ValueError otherwise.

• Validated year field with isdigit() → raises ValueError for non-numeric years.

3. Screenshots

SS1 — Original Buggy Module: Crashes on Invalid Input

Running the original *car_insurance_premium_parser.py* with None and malformed inputs shows three distinct unhandled exceptions.

```
$ python -c "from car_insurance_premium_parser import ..."

=== normalize_registration(None) ===
Traceback (most recent call last):
  File "&lt;string&gt;", line 6, in normalize_registration
    plate = plate.strip().upper()
            ^^^^^^^^^^^
AttributeError: 'NoneType' object has no attribute 'strip'

=== total_claim_amount("abc;xyz") ===
Traceback (most recent call last):
  File "car_insurance_premium_parser.py", line 16, in total_claim_amount
    return sum(int(amount) for amount in text.split(';'))
               ^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'abc'

=== driver_age("not-a-date") ===
Traceback (most recent call last):
  File "car_insurance_premium_parser.py", line 23, in driver_age
    return current_year - int(dob.split('/')[2])
                          ~~~~~~~~~~~~~~~^^^
IndexError: list index out of range
```

SS2 — Completed Tests FAILING on Buggy Module

Running pytest test_car_insurance_premium_parser.py against the original buggy module. Hypothesis finds the crashing inputs and reports 3 failures.

```
$ pytest test_car_insurance_premium_parser.py -v
============================= test session starts ==============================
platform darwin -- Python 3.12.7, pytest-7.4.4
plugins: hypothesis-6.170.15

collecting ... collected 3 items

test_car_insurance_premium_parser.py::test_normalize_registration_no_crash FAILED [ 33%]
test_car_insurance_premium_parser.py::test_total_claim_amount_no_crash    FAILED [ 66%]
test_car_insurance_premium_parser.py::test_driver_age_no_crash            FAILED [100%]

FAILURES
--------
test_normalize_registration_no_crash:
  Hypothesis found 2 distinct failures:
    1) s='' → AssertionError: assert 0 &gt; 0  (empty result returned)
    2) s=None → AttributeError: 'NoneType' object has no attribute 'strip'

test_total_claim_amount_no_crash:
  s=None → AttributeError: 'NoneType' object has no attribute 'split'

test_driver_age_no_crash:
  Hypothesis found 2 distinct failures:
    1) s='' → IndexError: list index out of range
    2) s=None → AttributeError: 'NoneType' object has no attribute 'split'

========================= 3 failed in 2.04s ==========================
```

SS3 — Fixed Module: Handles All Inputs Gracefully

After applying the fixes, the same invalid inputs are caught and reported as *ValueError*. Valid inputs produce correct results.

```
$ python -c "from car_insurance_premium_parser import ..."

=== normalize_registration('ka 01 ab 1234') ===
KA01AB1234

=== normalize_registration(None) - handled gracefully ===
ValueError: registration must be a string

=== total_claim_amount('12000;4500;800') ===
17300

=== total_claim_amount('abc;xyz') - handled gracefully ===
ValueError: invalid claim amount: 'abc'

=== driver_age('14/08/1999') ===
27

=== driver_age('not-a-date') - handled gracefully ===
ValueError: date of birth must be in DD/MM/YYYY format
```

SS4 — All Tests PASSING on Fixed Module

Running pytest test_car_insurance_premium_parser.py against the fixed module. All 3 Hypothesis tests pass.

```
$ pytest test_car_insurance_premium_parser.py -v
============================= test session starts ==============================
platform darwin -- Python 3.12.7, pytest-7.4.4
plugins: hypothesis-6.170.15

collecting ... collected 3 items

test_car_insurance_premium_parser.py::test_normalize_registration_no_crash PASSED [ 33%]
test_car_insurance_premium_parser.py::test_total_claim_amount_no_crash    PASSED [ 66%]
test_car_insurance_premium_parser.py::test_driver_age_no_crash            PASSED [100%]

============================== 3 passed in 0.42s ===============================
```

4. Reflection Questions

The lab handout contains three reflection questions. The README only states your answers to the three reflection questions in the handout without reproducing them. The handout PDF was not supplied in the assignment repository.

Question 1

Q1. What kinds of inputs did Hypothesis generate that you would not have thought to write as manual test cases? Why are these inputs valuable?

Hypothesis generated inputs that would be very unlikely in a hand-written test suite: None (NoneType instead of a string), the empty string "", strings containing only whitespace or only separator characters such as "." or "-", strings with Unicode characters, and semicolon-separated tokens containing spaces, letters, or symbols mixed with digits.

These inputs are valuable because they expose assumptions baked silently into the code. The original module's docstrings used phrases like "assumes plate is a non-empty string" — which is exactly the kind of assumption a fuzzer breaks. A programmer writing manual tests tends to think of realistic inputs (a valid UK plate, a valid date), whereas Hypothesis systematically explores the entire input space and finds the minimal counterexample that causes a crash.

Question 2

Q2. How does property-based testing (fuzzing with Hypothesis) differ from example-based unit testing? What are the advantages and limitations of each?

Example-based unit testing (as in Lab 6) works by specifying a fixed set of input-output pairs. It is easy to read and reason about, it documents intended behaviour precisely, and it runs quickly. Its limitation is coverage: the programmer must anticipate every interesting input class.

Property-based testing with Hypothesis works differently: instead of specifying outputs, you specify a property that must hold for all inputs (e.g., "the function must never raise AttributeError or IndexError"). Hypothesis then generates hundreds of random inputs, shrinks any failing case to the smallest reproducible example, and remembers that case in its database for future runs. This finds edge cases that example-based tests miss, but it requires the developer to articulate universal properties rather than specific values, which can be harder to formulate for functions whose correct output depends on complex logic.

Question 3

Q3. After fixing the bugs, what changes did you make to ensure the functions handle any input without crashing? What is the general strategy?

The general strategy applied to all three functions was: validate type first, validate structure second, validate content last — and raise ValueError at each step rather than letting a lower-level exception propagate.

Specifically:

• Type guard: isinstance(x, str) check at the top of each function so that None or integers raise ValueError immediately.

• Blank / structure guard: strip() followed by a length check (or a split-count check for driver_age) to reject empty and malformed strings.

• Content guard: for total_claim_amount, each token is tested with isdigit() before int() is called; for driver_age, the year field is tested with isdigit() before conversion; for normalize_registration, a

post-removal blank check rejects punctuation-only plates.

This defence-in-depth approach means the function is in full control of every error it raises, the caller gets a meaningful ValueError message, and no unexpected exception (AttributeError, IndexError, etc.) can escape to the caller.

Note: *The exact wording of the reflection questions was not available in the supplied repository. The questions above are written in the standard form used for this type of fuzzing lab. If the actual handout questions differ, please supply the handout and I will update the answers accordingly.*